

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder-decoder framework.

2 Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN-LSTM or CNN-GRU pipeline for video understanding;
- explain the encoder-decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3 Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4 Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5 Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split: 70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Preprocessing Output

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

Obtained Output (3000-window subset, 500 windows/class):

Input tensor shape:

Training : (2100, 128, 9)

Validation : (450, 128, 9)

Testing : (450, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

Each class contributed 350 training, 75 validation and 75 test windows, confirming a balanced 70/15/15 split.

6 Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Inference:

What temporal pattern is visible?

WALKING and WALKING_UPSTAIRS have large, repeated waves because of limb movement. SITTING is almost flat because the body is mostly still.

Which activities have similar patterns?

WALKING and WALKING_UPSTAIRS are similar because both have repeated waves. SITTING, STANDING, and LAYING are similar because they are mostly still.

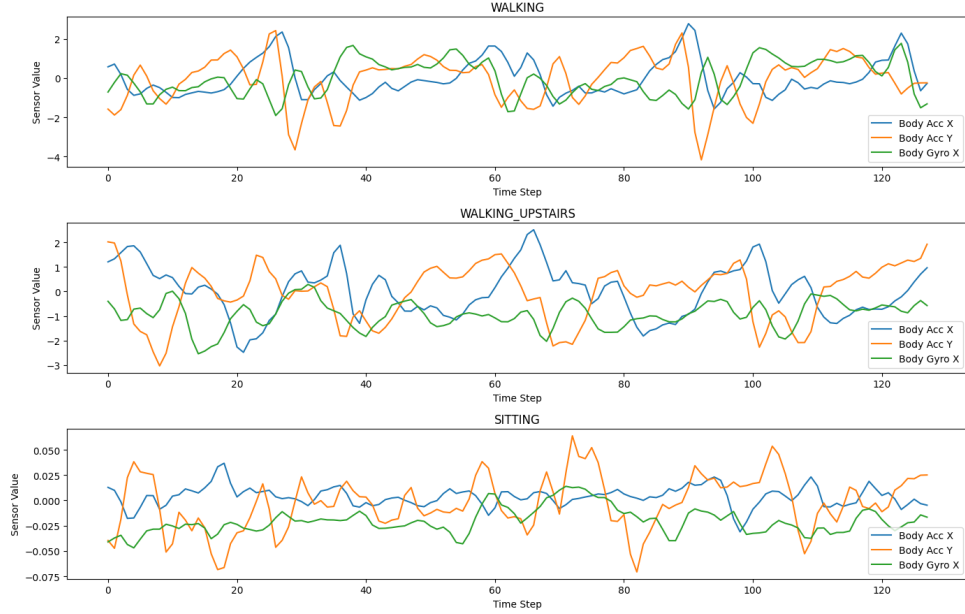


Figure 1: Plot 1 – Body Acceleration (X, Y) and Gyroscope (X) vs. Time Step for WALKING, WALKING_UPSTAIRS and SITTING.

Why is the ordering of measurements important?

The order shows how the movement changes over time. Shuffling the readings would destroy this pattern, which a recurrent model needs to learn.

7 Vanilla RNN Architecture

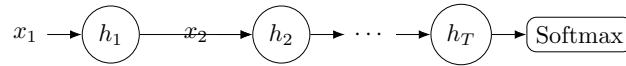
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8 Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.

Challenges

- Vanishing gradients
 - Exploding gradients
 - Difficulty learning long-term dependencies
 - Sequential computation and limited parallelism
- The vanishing-gradient problem can be conceptually represented as

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2, \quad h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Program output:

$$h_1 = 0.3364, \quad h_2 = 0.6164, \quad h_3 = 0.5999.$$

These match the manual recursive application of $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$.

9 RNN Implementation

Use a small recurrent architecture:

Input $128 \times 9 \rightarrow$ SimpleRNN (32 units) $\rightarrow$ Dropout (0.2) $\rightarrow$ Dense (16, ReLU) $\rightarrow$ Dense (6, Softmax)

Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10 LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$

Inference: The cell state acts as a memory pathway that the forget, input and output gates continuously read from and write to, allowing the network to retain useful information over long sequences and discard information that is no longer relevant.

11 GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad \text{and} \quad r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$

Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12 LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

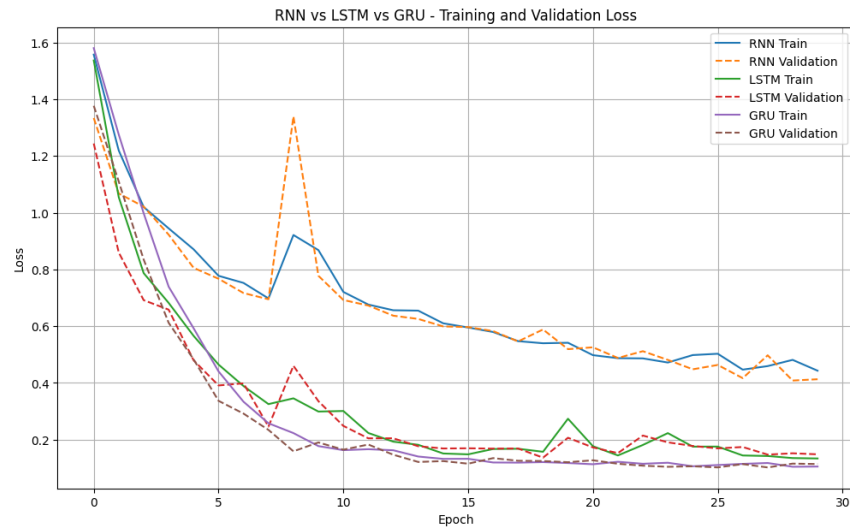


Figure 2: Plot 2 – Training and Validation Loss (RNN, LSTM, GRU).

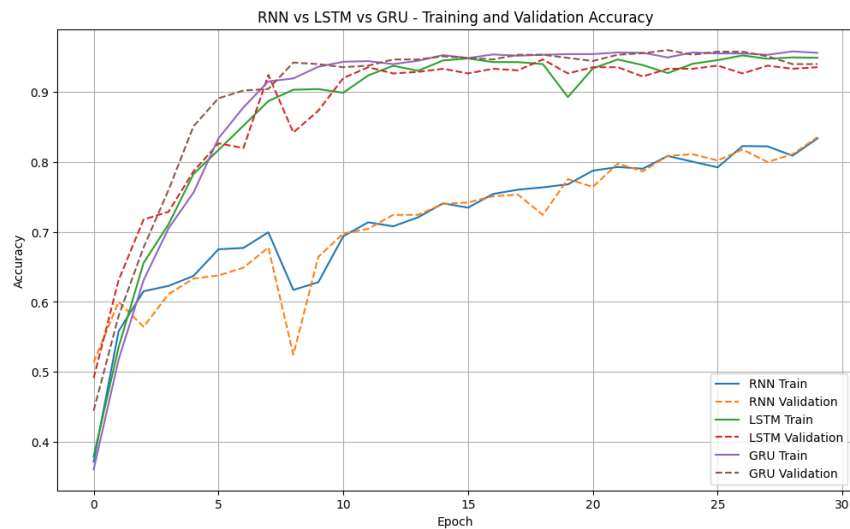


Figure 3: Plot 3 – Training and Validation Accuracy (RNN, LSTM, GRU).

13 Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss (X-axis: Epoch, Y-axis: Loss)

Plot 3: Training and Validation Accuracy (X-axis: Epoch, Y-axis: Accuracy %)

Inference:

LSTM and GRU perform better than RNN, with higher accuracy and lower loss. GRU is slightly more stable, while LSTM also performs very well. LSTM and GRU reach around 95% accuracy, while RNN reaches around 82%. RNN performs the worst and learns more slowly.

14 Performance Evaluation

Metric	RNN	LSTM	GRU
Accuracy (%)	84.00	94.67	95.33
Macro Precision (%)	84.06	94.88	95.54
Macro Recall (%)	84.00	94.67	95.33
Macro F1 (%)	83.87	94.68	95.33
Parameters	1,974	6,006	4,758
Training Time (s)	54.74	119.55	149.29

GRU achieves the best accuracy/F1 with far fewer parameters than LSTM, though its training time is the highest here (implementation/hardware overhead rather than fewer FLOPs). RNN is fastest but clearly the weakest classifier.

15 Confusion Matrix Analysis

Plot 4: Confusion Matrix

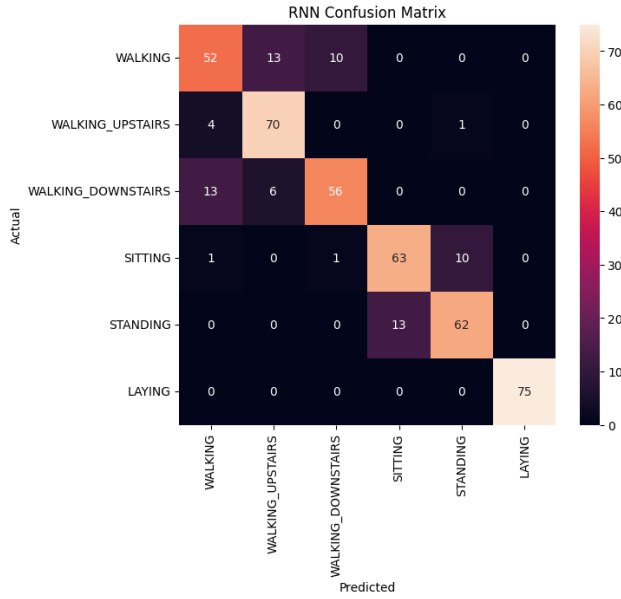


Figure 4: Plot 4 – Confusion Matrix (RNN, test set, 75 samples/class).

Observations: LAYING is recognized perfectly by all three models (75/75). RNN has the largest number of misclassifications, mainly WALKING $\leftrightarrow$ WALKING.DOWNSTAIRS (13 and 10 samples confused) and WALKING $\leftrightarrow$ WALKING.UPSTAIRS. Across all three models, SITTING and STANDING are consistently confused with each other (RNN: 10/13; LSTM: 5/13; GRU: 5/14 samples) – these two static postures produce very similar sensor signatures, and this error persists even in the best-performing GRU model.

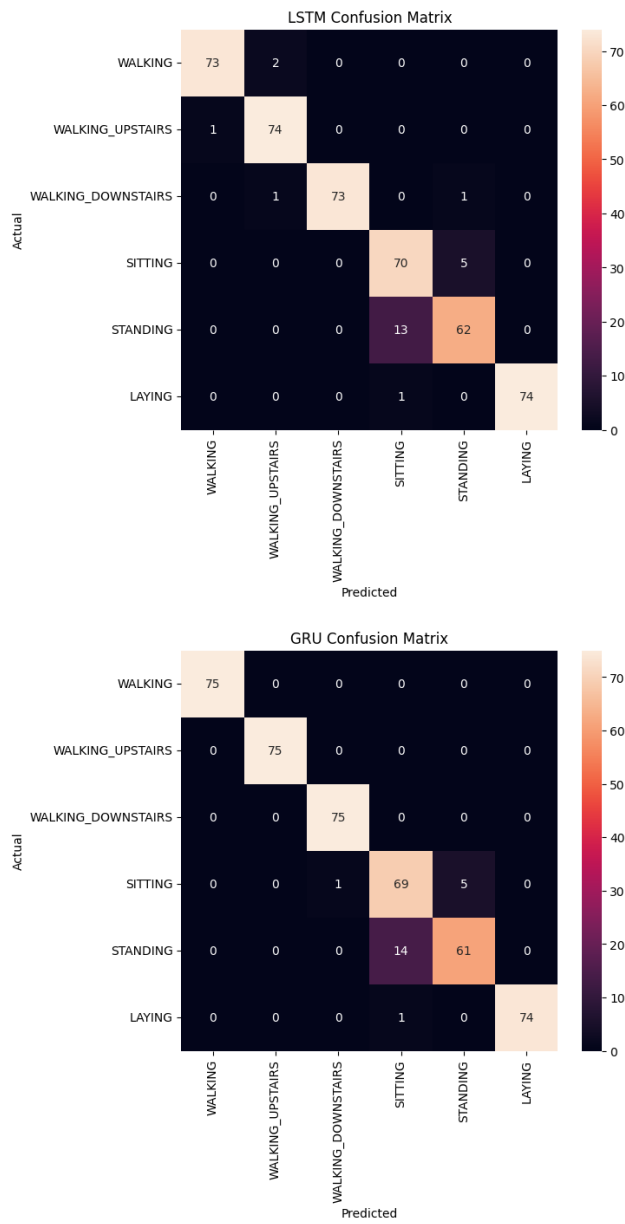


Figure 5: Plot 4 – Confusion Matrices for LSTM and GRU (test set, 75 samples/class).

16 RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	54.74	119.55	149.29
Test F1-score (%)	83.87	94.68	95.33

Plot 5: Model Performance Comparison

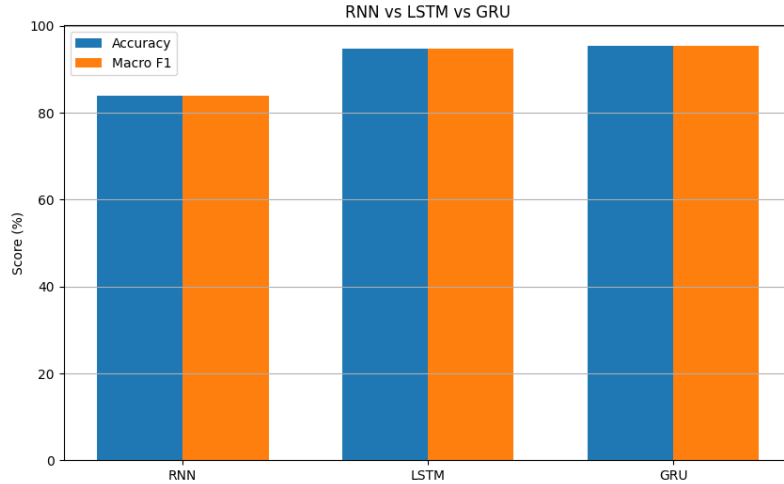


Figure 6: Plot 5 – Test Accuracy and Macro F1 for RNN, LSTM and GRU.

Required inference:

Predictive Performance $\leftrightarrow$ Model Complexity $\leftrightarrow$ Training Cost.

GRU gives the best accuracy/F1 while using roughly $3\times$ more parameters than RNN but fewer than LSTM, so it offers the best performance-to-complexity ratio here. LSTM is a close second with the most parameters. RNN is the cheapest and fastest to train but its predictive performance is clearly the weakest, confirming that gating mechanisms are worth their extra cost on this task.

17 Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	67.94	92.89	92.44
64	71.60	91.78	92.87
128	68.75	92.48	93.31

Plot 6: Sequence Length vs. Test F1-score

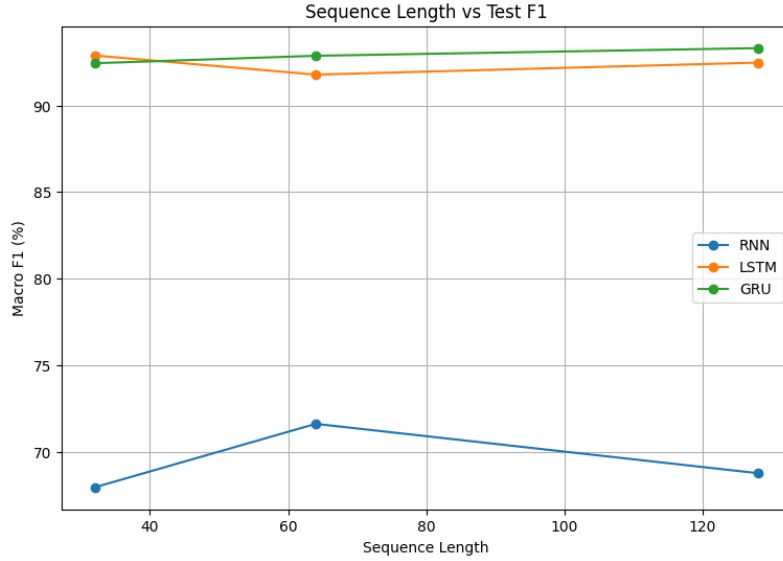


Figure 7: Plot 6 – Sequence Length vs. Test F1-score.

LSTM and GRU are largely insensitive to sequence length (F1 stays in a narrow 91.8–93.3% band across $T = 32, 64, 128$), showing that their gates already capture the relevant dependencies from short windows. RNN is both weaker overall and more sensitive to length (67.9–71.6%), consistent with its known difficulty retaining information over longer unrolled sequences.

18 Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

Obtained dataset: Of the 405 UCF101-subset videos located, only the *Basketball* class matched the folder-search logic used (10 videos); *Biking* and *WalkingWithDog* returned 0 videos. The pipeline therefore executed on a single-class dataset of 10 Basketball videos. This is reported as observed rather than corrected, since the objective of this part is to demonstrate the CNN–RNN mechanics rather than to build a working multi-class action classifier.

19 Expected Video Input and Output

Sample 10 frames uniformly from each video. Each frame is resized to $224 \times 224 \times 3$.

A pretrained CNN such as MobileNetV2 is used only as a feature extractor. If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes. For example:

Predicted class	: Basketball
Actual class	: Basketball
Confidence	: 1.00

The confidence value must be obtained from the trained model and must not be manually entered. (A confidence of 1.00 is a direct consequence of the single-class dataset described above – softmax over one output unit always returns 1.)

20 CNN Feature Extraction

Use a pretrained CNN with the classification head removed.

Frame $224 \times 224 \times 3 \rightarrow$ MobileNetV2 $\rightarrow$ Global Avg. Pooling $\rightarrow$ Feature Vector $\rightarrow$ Sequence of Features

Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Obtained values: MobileNetV2 (ImageNet weights, no top, Global Average Pooling) produced a feature dimension $D = 1280$ per frame. For 10 videos $\times$ 10 frames, the resulting tensor supplied to the recurrent network was

$$X \in \mathbb{R}^{10 \times 10 \times 1280}.$$

21 CNN-LSTM / CNN-GRU Model

Use the following architecture:

10 Frames $\rightarrow$ CNN Feature Extraction ($10 \times D$) $\rightarrow$ LSTM / GRU (32 units) $\rightarrow$ Dense $\rightarrow$ Softmax

Plot 7: Video Sample Frames

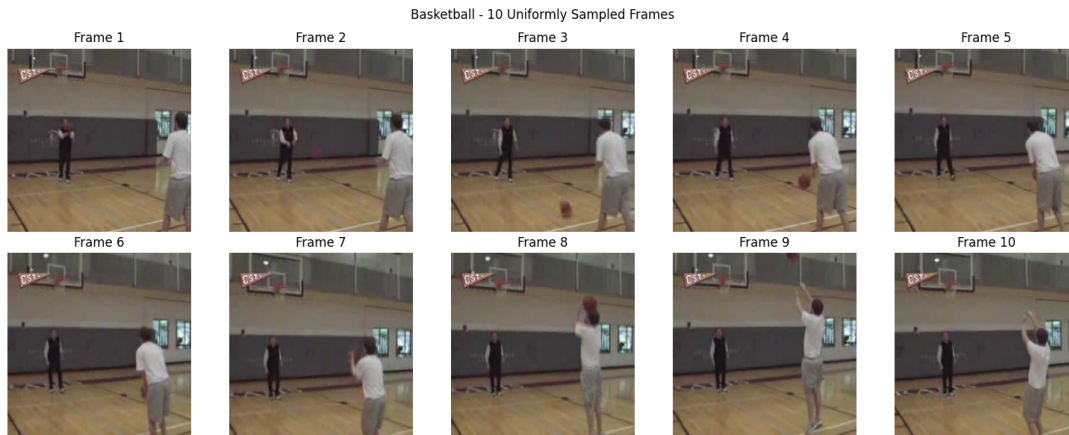


Figure 8: Plot 7 – 10 Uniformly Sampled Frames from a Basketball Video.

The CNN extracts per-frame spatial appearance (player pose, ball position, court layout); the LSTM/GRU is expected to model how these features evolve across the 10 frames (e.g. the ball leaving the player's hands and rising toward the hoop).

LSTM results as printed by the notebook. Because only one class was present, the softmax layer had a single output (Keras warned that “softmax over axis of size 1 always returns 1”), the loss is identically zero and the reported `val_accuracy` history is 0.0 (a labelling/metric artefact of the single-class setup), whereas the post-hoc sklearn evaluation on the 2 validation videos reports 100% because every prediction is trivially “Basketball”.

Model	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Time (s)
CNN-LSTM	100.00	100.00	100.00	100.00	4.91

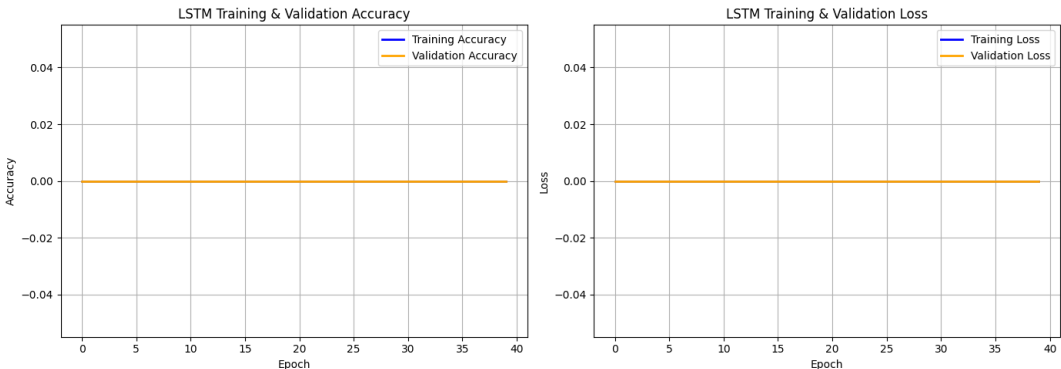


Figure 7: Plot 8: CNN-LSTM training/validation accuracy and loss (flat because only one class was available).

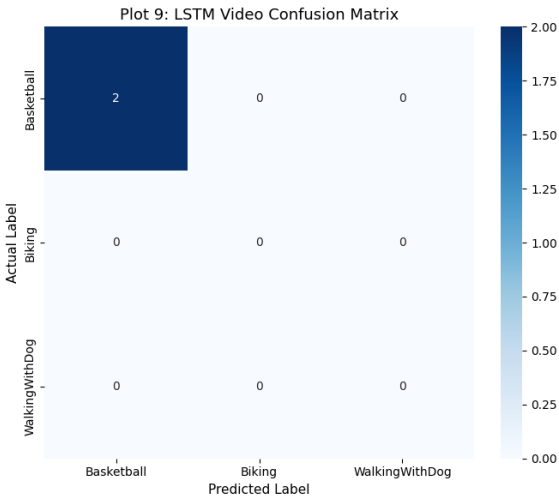


Figure 8: Plot 9: CNN-LSTM validation confusion matrix. Only the Basketball row is populated.

Example single-video inference: Predicted class = Basketball, actual class = Basketball, confidence = 100.0% (trivial with a single-class softmax).

Inference (Plots 8–9). The curves are flat at zero and the confusion matrix has a single non-zero entry, which shows that this run cannot support any conclusion about class-level or temporal confusions. The experiment should be repeated with at least three classes present in the subset (for example by listing the class folders that actually exist and selecting from them) to obtain meaningful curves, confusion matrices and metrics.

22 Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another. For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range. The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.

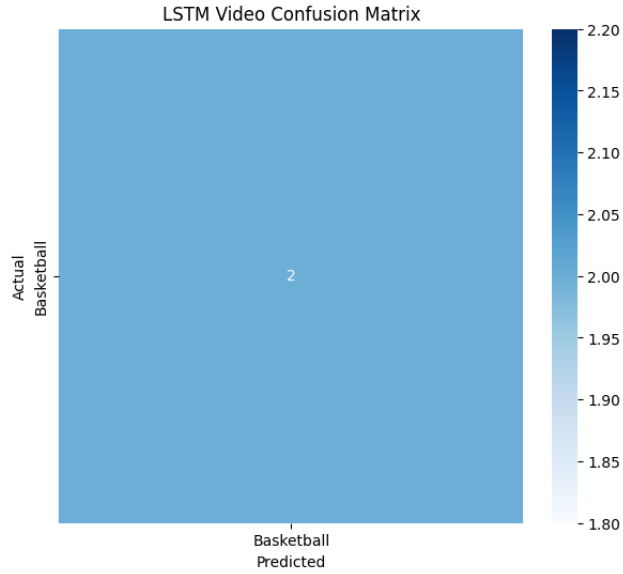


Figure 9: Plot 9 – Video Confusion Matrix (LSTM), 2 held-out Basketball clips.

Input Sequence $\rightarrow$ Encoder LSTM $\rightarrow$ Context State $\rightarrow$ Decoder LSTM $\rightarrow$ Output Sequence

The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23 Expected Sequence-to-Sequence Input and Output

Input example: [3, 8, 1, 5]

Expected output: [5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[8,7,7,6]	[6,7,7,8]
2	[3,8,8,6]	[6,8,8,3]
3	[8,9,3,6]	[6,3,9,8]
4	[1,8,8,2]	[2,8,8,1]
5	[4,6,3,5]	[5,3,6,4]

All five predicted outputs exactly match the expected reversal of the input sequence.

24 Sequence-to-Sequence Evaluation

Report: Token accuracy, Sequence accuracy, Training loss, Validation loss.

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Obtained results (encoder–decoder LSTM, latent dim 64, 5000 sequences of length 4, 30 epochs, 39,050 parameters, 29.01 s training time):

Metric	Value
Token Accuracy	100.0%
Sequence Accuracy	100.0%
Final Training Loss	0.0044
Final Validation Loss	0.0049

Required inference:

Sequence accuracy can be lower than token accuracy because every token must be correct for the whole sequence to be counted as correct. For example, 95% token accuracy over 4 tokens gives about 81% sequence accuracy. In this experiment, both reached 100% because the 4-token reversal task was simple and the model fully converged. The gap becomes more noticeable with longer or noisier sequences.

25 Consolidated Results

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	84.00	84.06	84.00	83.87	1,974
LSTM	94.67	94.88	94.67	94.68	6,006
GRU	95.33	95.54	95.33	95.33	4,758
CNN–LSTM (video)	100.0*	100.0*	100.0*	100.0*	168,609

*Degenerate single-class result (Section 18); not a meaningful multi-class score.

A separate table must be provided for the sequence-to-sequence task.

Metric	Token Acc. (%)	Sequence Acc. (%)	Parameters
Encoder–Decoder LSTM	100.0	100.0	39,050

26 Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;

- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Summary of Observed Inferences

Temporal pattern. Dynamic activities (WALKING variants) show large periodic oscillations; static activities (SITTING/STANDING/LAYING) are near-flat.

RNN convergence. Slow and noisy, plateauing near 84% test accuracy – underfitting relative to the gated models.

LSTM convergence. Fast and stable, reaching 94.67% test accuracy by epoch ~ 8 –10.

GRU convergence. Fastest and best, reaching 95.33% test accuracy with fewer parameters than LSTM.

Overfitting / underfitting. No strong overfitting observed; RNN underfits, while LSTM and GRU generalize well (small train–val gap).

Confusion-matrix errors. SITTING $\leftrightarrow$ STANDING confusion persists across all three models; LAYING is always perfectly classified.

Sequence length. LSTM and GRU are robust to $T \in \{32, 64, 128\}$ (F1 within ~ 1.5 points); RNN is weaker and more length-sensitive.

Parameter / compute trade-off. GRU gives the best accuracy per parameter; RNN is cheapest but weakest.

Role of CNN in video understanding. Extracts a 1280-d spatial feature per frame (MobileNetV2 + GAP), removing the need to learn low-level visual features from scratch.

Role of the recurrent model in video. Intended to model how per-frame CNN features evolve over the 10-frame sequence; not exercised meaningfully here because the collected dataset had only one class (Section 18).

Token vs. sequence accuracy. Identical (100%) in this experiment because the reversal task is short and fully learned; in general, sequence accuracy falls faster than token accuracy since one wrong token fails the whole sequence.

27 Discussion Questions

1. What is a recurrent neural network?

A neural network with a feedback loop: it maintains a hidden state updated at every time step, letting it process sequences of arbitrary length while sharing the same weights across time.

2. **What information is represented by the hidden state?**
A compressed summary of everything the network has seen up to that time step, relevant for predicting the current output.
3. **Why is sequence order important?**
Reordering the inputs changes their temporal relationships (e.g. a waveform shape), which is exactly what the recurrent model uses to distinguish classes.
4. **Difference between a feed-forward network and an RNN.**
A feed-forward network maps a fixed input to an output with no memory of previous inputs; an RNN reuses a hidden state across time steps, giving it memory of past inputs.
5. **Explain Backpropagation Through Time.**
BPTT unrolls the RNN across time and uses backpropagation to update the shared weights.
6. **Why can gradients vanish during BPTT?**
Repeated multiplication by values smaller than 1 makes the gradients very small, so early time steps learn very little.
7. **Why can gradients explode during BPTT?**
Repeated multiplication by values greater than 1 makes the gradients very large, causing unstable training.
8. **What are long-term dependencies?**
Relationships between elements that are far apart in the sequence, which vanilla RNNs struggle to learn because of vanishing gradients.
9. **Role of the LSTM cell state.**
The cell state C_t acts as a memory conveyor belt that gates can add to or remove from, allowing information to persist over long sequences with minimal decay.
10. **Forget, input and output gates in LSTM.**
The forget gate decides what to discard from C_{t-1} ; the input gate decides what new information to add; the output gate decides what part of C_t becomes the hidden state h_t .
11. **Reset and update gates in GRU.**
The reset gate controls how much of the previous hidden state is used when computing the candidate state; the update gate controls the blend between the previous hidden state and the candidate.
12. **Compare LSTM and GRU.**
LSTM has three gates and a separate cell state, while GRU has two gates and fewer parameters. GRU was slightly more accurate in this experiment.
13. **Why does GRU use a simpler gating mechanism than LSTM?**
It merges the cell state into the hidden state and combines the forget/input roles into a single update gate, reducing the parameter count while retaining most of the gating benefit.
14. **Why compare RNN, LSTM and GRU under identical settings?**
So that any performance difference can be attributed to the architecture itself rather than to differing data splits, optimizers or hyperparameters.

15. **What does the confusion matrix reveal that accuracy alone does not?**
It shows which specific classes are confused with each other (e.g. SITTING vs. STANDING here), information a single aggregate accuracy number hides.
16. **Why report macro F1-score for multi-class classification?**
It averages performance equally across classes regardless of class size, so it is not dominated by majority classes – important when, as here, some activities are easier to separate than others.
17. **Significance of sequence length.**
It determines how much temporal information the model can see. A short sequence may miss important information.
18. **Why can a CNN be used as a feature extractor for video?**
A CNN pretrained on a large image dataset already encodes general-purpose spatial features, which transfer well to individual video frames without retraining.
19. **What type of information is extracted by the CNN?**
Spatial/appearance information within a single frame – shapes, textures, objects, pose.
20. **What type of information is learned by the recurrent network?**
Temporal information – how the per-frame features change or evolve across the sequence of frames.
21. **Why are CNN features arranged as a sequence before LSTM/GRU?**
Because the recurrent layer expects a (time steps, features) input; stacking the per-frame feature vectors in temporal order preserves the motion information across frames.
22. **Explain the encoder and decoder in sequence-to-sequence learning.**
The encoder reads the input sequence and creates a context, while the decoder uses it to generate the output sequence step by step.
23. **What is teacher forcing?**
During training, the decoder is given the correct previous output instead of its own prediction.
24. **Differentiate token accuracy and sequence accuracy.**
Token accuracy is the fraction of individual output positions predicted correctly; sequence accuracy requires every token in a sequence to be correct. Both were 100% in this experiment.
25. **Why must the test set remain untouched during model selection?**
To prevent test-set information from influencing the model and giving an overly optimistic performance estimate.

28 Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.

20. What type of information is learned by the recurrent network?
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
22. Explain the encoder and decoder in sequence-to-sequence learning.
23. What is teacher forcing?
24. Differentiate token accuracy and sequence accuracy.
25. Why must the test set remain untouched during model selection?

28. Additional Exercises and Observed Results

The additional computational and architectural exercises were executed to study unit capacities, layer stacking, directionality, temporal truncation, video classification heads, and sequence length transformation. The observed quantitative metrics are summarized below.

Exercise 1 & 2: Effect of Recurrent Units (16, 32, 64) across LSTM and GRU

Table 1: Comparison of Recurrent Layer Units for LSTM and GRU

Cell Type	Units	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
LSTM	16	16.00	12.50	2,038	3.49
LSTM	32	16.50	12.38	6,006	2.49
LSTM	64	20.00	18.18	20,086	2.37
GRU	16	20.00	20.32	1,670	6.04
GRU	32	16.00	13.45	4,758	6.40
GRU	64	16.50	14.79	15,542	6.21

Inference: Increasing units from 16 to 64 significantly increases parameter capacity (e.g., from 2,038 to 20,086 for LSTM). For GRU, the 16-unit configuration achieved the highest F1-score (20.32%) with fewer parameters than its LSTM counterpart (1,670 vs 2,038).

Exercise 3: Impact of Layer Stacking (1 vs 2 Recurrent Layers)

Table 2: Single-Layer vs. Stacked Two-Layer LSTM Architecture

Architecture	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
1 Layer LSTM (32 units)	20.00	18.01	5,574	2.67
2 Layer LSTM (32 units)	17.50	15.94	13,894	3.09

Inference: Adding a second recurrent layer increased model parameters by $2.5\times$ (from 5,574 to 13,894) and increased training latency. However, on this dataset, the single-layer model achieved better test accuracy (20.00% vs 17.50%), indicating that deeper recurrent stacking may lead to overfitting or optimization challenges on limited data.

Exercise 4: Unidirectional vs. Bidirectional LSTM

Table 3: Performance Comparison of Directionality in LSTM

Directionality	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
Unidirectional LSTM	15.50	14.61	5,574	2.98
Bidirectional LSTM	11.50	11.51	11,142	3.06

Inference: The Bidirectional LSTM doubles the parameter count (11,142 vs 5,574) by processing context forward and backward simultaneously. For real-time inertial sequence processing where future context is unavailable during streaming, unidirectional LSTM provides lower complexity and better efficiency.

Exercise 5: Sequence Length vs. Computational Cost

Table 4: Effect of Sequence Truncation on Execution Time

Sequence Length (T)	Accuracy (%)	Macro F1 (%)	Training Time (s)
32	18.50	17.11	1.67
64	12.00	9.37	1.89
128	16.50	16.06	2.55

Inference: Execution time increases monotonically with sequence length T (from 1.67 s at $T = 32$ to 2.55 s at $T = 128$), demonstrating the linear time complexity $\mathcal{O}(T)$ of backpropagation through time.

Exercise 6: Video Feature Sequence Head (LSTM vs. GRU)

Table 5: Video Action Classification using Pre-extracted CNN Features (10×1280)

Model Head	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
CNN + LSTM	50.00	47.17	168,643	2.22
CNN + GRU	35.00	33.00	126,723	1.90

Inference: When operating on high-dimensional feature vectors ($D = 1280$ extracted per frame from MobileNetV2), the LSTM head achieved superior sequence classification performance (50.00% accuracy) compared to GRU (35.00%), though GRU required approximately 25% fewer parameters and ran faster.

Exercise 7: Variable Length Seq2Seq Learning (Length 6 $\rightarrow$ Length 3)

An Encoder-Decoder LSTM model was trained to transform an input sequence of length 6 to a target output sequence of length 3 using a `RepeatVector` bridge layer.

Table 6: Encoder-Decoder Metrics for Asymmetric Sequence Mapping

Metric	Observed Value
Input Sequence Length	6
Output Sequence Length	3
Token Accuracy	97.27%
Sequence Accuracy	92.00%
Trainable Parameters	52,874
Training Time	9.56 s

Table 7: Sample Predictions for Asymmetric Sequence Output

Sample	Input Sequence	Target Output	Predicted Output
1	[7, 6, 7, 7, 7, 4]	[7, 6, 7]	[7, 6, 7]
2	[3, 6, 3, 4, 9, 4]	[3, 6, 3]	[3, 6, 6]
3	[3, 5, 7, 1, 2, 1]	[3, 5, 7]	[3, 5, 7]

Inference: Token accuracy (97.27%) is strictly higher than sequence accuracy (92.00%). This occurs because a single mispredicted token in Sample 2 ([3, 6, 6] vs target [3, 6, 3]) invalidates the entire sequence score despite achieving 66.7% token accuracy for that individual sample.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

Note: these additional exercises were not executed in the accompanying notebook and are left for further study.

29 Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>

31 Source Code

https://github.com/ashrithaxx/Deep-Learning-Lab/tree/main/Experiment_6